

Fluent API in Entity Framework Core

In this article, I will discuss How to Implement **Fluent API Configurations in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [ConcurrencyCheck Attribute in Entity Framework Core](#) with Examples. Like the Data Annotation Attribute, the Entity Framework Core provides Fluent API, which we can also use to configure the domain classes, which will override the default conventions that the Entity Framework Core follows.

What is Fluent API in Entity Framework Core?

In Entity Framework Core (EF Core), Fluent API (based on the Fluent API/Interface Design Pattern) is a way to configure entity classes and relationships in a more flexible manner. It offers a fluent interface, meaning that we chain multiple methods together to set up configurations directly in code rather than using data annotations. It provides a powerful approach to customize how EF Core maps entity classes to the underlying database schema, making it suitable for complex entity setups where data annotations might fall short.

Unlike Data Annotations, which use attributes in the entity class, Fluent API configurations are defined within the `OnModelCreating` method of the `DbContext` class via the `ModelBuilder` object using a fluent (method-chaining) syntax.

What is the Fluent Interface Design Pattern?

The Fluent Interface Design Pattern is a software design pattern that allows method chaining to create more readable, concise, and expressive code. It focuses on improving the readability of the code by designing methods in such a way that they can be chained together in a flow-like manner. This pattern is commonly used in APIs, including Entity Framework Core's Fluent API, LINQ, and libraries like AutoMapper.

Example to Understand Fluent Interface Design Pattern:

The main objective of the Fluent Interface Design Pattern is to apply multiple methods to an object by connecting them with dots (.) without having to re-specify the object name each time. Let's understand how to Implement a Fluent Interface Design Pattern with an example. Let's say we have the following Student class.

```
public class Student
{
    public string RegdNo { get; set; }
    public string Name { get; set; }
    public DateTime DOB { get; set; }
    public string Branch { get; set; }
    public string Address { get; set; }
}
```

Now, if we want to consume the above Student class, we generally create an instance of it and set its respective properties, as shown in the image below.

```
Student student = new Student();
student.Name = "Pranaya Rout";
student.RegdNo = "BQPPR123456";
student.DOB = Convert.ToDateTime("10/10/1992");
student.Branch = "CSE";
student.Address = "BBSR, Odisha";
```

The Fluent Interfaces Design Pattern simplifies our object consumption code by making it simpler, readable, and understandable. Would it not be nice to set the Student object properties as shown in the image below?

```
Student student = new Student();
student.StudentRegedNumber("BQPPR123456")
    .NameOfTheStudent("Pranaya Rout")
    .BornOn("10/10/1992")
    .StudyOn("CSE")
    .StaysAt("BBSR, Odisha")
```

Consuming the object like the above interface is like speaking a sentence that would make the class consumption code more simple, readable, and understandable. The next big thing we must understand is how to implement this. To implement this, we have something called Method Chaining.

What is Method Chaining?

Method Chaining is a technique or process where multiple methods are called sequentially on an object within a single statement. This is achieved because each method returns the object itself (this), allowing the following method to be called on that object.

Let us understand how to implement Method Chaining. To implement method chaining in C#, we first need to create a wrapper class around the entity class. In our case, the entity class is Student, so we need to create a Wrapper class around the Student class, as shown in the image below.

```

public class FluentStudent
{
    private Student student = new Student();
    public FluentStudent StudentRegedNumber(string RegdNo)
    {
        student.RegdNo = RegdNo;
        return this;
    }
    public FluentStudent NameOfTheStudent(string Name)
    {
        student.Name = Name;
        return this;
    }
    public FluentStudent BornOn(string DOB)
    {
        student.DOB = Convert.ToDateTime(DOB);
        return this;
    }
    public FluentStudent StudyOn(string Branch)
    {
        student.Branch = Branch;
        return this;
    }
    public FluentStudent StaysAt(string Address)
    {
        student.Address = Address;
        return this;
    }
}

```

As you can see in the above FluentStudent class, we have created methods for each Student property. First, we create an instance of Student Class, and then, in each method, we set the value of the respective Student Property.

Further, notice that each method's return type is set to FluentStudent, which is essential; because of this, we can call subsequent methods using the dot operator. Now, the client will consume the above fluent interface. So, with the above FluentStudent wrapper class in place, the client code looks as shown below.

```

FluentStudent student = new FluentStudent();
student.StudentRegedNumber("BQPPR123456")
    .NameOfTheStudent("Pranaya Rout")
    .BornOn("10/10/1992")
    .StudyOn("CSE")
    .StaysAt("BBSR, Odisha")

```

Complete Example code of Fluent Interface Design Pattern:

Whatever we have discussed so far is given in the below example. The following example shows how to implement the Fluent Interface Design Pattern in C#.

```
using System;
namespace FluentInterfaceDesignPattern
{
    public class Program
    {
        static void Main(string[] args)
        {
            FluentStudent student = new FluentStudent();
            student.StudentRegedNumber("BQPPR123456")
                .NameOfTheStudent("Pranaya Rout")
                .BornOn("10/10/1992")
                .StudyOn("CSE")
                .StaysAt("BBSR, Odisha");
            Console.Read();
        }
    }
    public class Student
    {
        public string RegdNo { get; set; }
        public string Name { get; set; }
        public DateTime DOB { get; set; }
        public string Branch { get; set; }
        public string Address { get; set; }
    }
    public class FluentStudent
    {
        private Student student = new Student();
        public FluentStudent StudentRegedNumber(string RegdNo)
        {
            student.RegdNo = RegdNo;
            return this;
        }
        public FluentStudent NameOfTheStudent(string Name)
        {
            student.Name = Name;
            return this;
        }
        public FluentStudent BornOn(string DOB)
        {
            student.DOB = Convert.ToDateTime(DOB);
            return this;
        }
        public FluentStudent StudyOn(string Branch)
        {
            student.Branch = Branch;
            return this;
        }
        public FluentStudent StaysAt(string Address)
```

```

    {
        student.Address = Address;
        return this;
    }
}

```

Now, I hope you understand the Fluent Interface Design Pattern. With this kept in mind, let us proceed and try to understand Fluent API in Entity Framework Core.

Configuring Fluent API in Entity Framework Core:

To configure the Fluent API in EF Core, we need to override the `OnModelCreating` method of the `DbContext` class and use the `ModelBuilder` object to define configurations for entities, properties, relationships, etc., something like the one shown in the image below. As you can see, we call many methods using the same `modelBuilder` object using the dot (.) operators, which is nothing but method chaining.

```

public class EFCoreDbContext : DbContext
{
    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) ...

    0 references
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        //Configure Domain Class using modelBuilder
        //Configuring the DateOfBirth Property of Student Domain Class
        modelBuilder.Entity<Student>()
            .Property(std => std.DateOfBirth)
            .HasColumnName("DOB")
            .HasColumnOrder(3)
            .HasColumnType("datetime2");
    }

    3 references
    public DbSet<Student> Students { get; set; }
}

```

Note: You need to remember that, in Entity Framework Core, we can configure a domain class using both Data Annotation Attributes and Fluent API simultaneously. In that case, EF Core will give precedence to Fluent API over Data Annotations Attributes.

Fluent API Configurations in Entity Framework Core:

In EF Core, the Fluent API is used to configure various aspects of model classes. Fluent API configurations are broadly divided into three categories. They are as follows:

1. **Model-Wide or Global Configuration**
2. **Entity Configuration**
3. **Property Configuration**

Model-Wide or Global Configurations in EF Core

Model-wide configuration (or Global Configuration) refers to the settings applied across the entire EF Core model. These configurations affect the behavior of all entities within a given DbContext. Global Configurations are applied inside the OnModelCreating method of our DbContext class. This type of configuration is used for things like global filters, conventions, or default behaviors that need to be consistent throughout the entire application. Some of the examples of [Global Configurations using Fluent API](#) include:

- **Setting the Default Schema for All Tables**
- **Setting Default Decimal Precision Globally**
- **Setting a Default Max Length for All String Properties**
- **Converting Enum Properties to Strings Globally**
- **Configuring Cascade Delete Behavior Globally**

Entity Configurations in EF Core:

Entity Configuration involves configuring specific settings for individual entities. This allows us to define how a particular entity is mapped to a table in the database, including specifying relationships, keys, indexes, and constraints. Entity-level configurations are typically done using the Entity<TEntity> method in the OnModelCreating method. Some of the examples of [Entity Configuration using Fluent API](#) include:

- **Configuring Table Names**
- **Configuring Primary Keys**
- **Configuring Composite Primary Keys**
- **Configuring Indexes**
- **Configuring Relationships (One-to-One, One-to-Many, Many-to-Many)**
- **Configuring Cascade Delete Behavior for Specific Relationships**
- **Ignoring Entities**
- **Configuring Alternate Keys (Unique Constraints)**

Property Configurations in Entity Framework:

Property Configuration involves configuring how individual properties of an entity are mapped to columns in the database. It allows us to specify details like data types, constraints (e.g., NOT NULL), default values, and computed columns. Property-level configurations can also be done using the OnModelCreating method. It allows detailed customization of how properties are treated in the database. Some of the examples of [Property Configuration using Fluent API](#) include:

- **Configuring Column Names**
- **Configuring Data Types**
- **Configuring Default Values**
- **Configuring Nullable and Required Properties**
- **Configuring Maximum Length**
- **Configuring Precision and Scale**
- **Configuring Computed Columns**
- **Configuring Value Conversions**
- **Configuring Concurrency Tokens**
- **Configuring Shadow Properties**
- **Ignoring Properties**

Note: In our upcoming article, I will explain all the above Fluent API Configuration with Examples using Entity Framework Core.

Summary:

Type	Scope	Example Configuration
Model-Wide (Global)	Entire model or schema	Default schema, query filters, cascade behavior.
Entity Configuration	Specific entity	Table name, primary key, relationships, indexes.
Property Configuration	Specific property of an entity	Column name, data type, constraints, defaults.

These configurations are typically done in the `OnModelCreating` method using the Fluent API, providing more control over how our application interacts with the database.

When Should We Use Fluent API Configurations in Entity Framework Core?

Fluent API configurations in EF Core are useful in scenarios where data annotations are insufficient or flexible. The following are some common cases when Fluent API is the preferred method:

- **Complex Configurations:** Fluent API should be used when the model requires complex configurations that cannot be easily achieved with data annotations.
- **Multiple Property Configurations:** When several configurations are needed for a single property (such as setting data types, constraints, default values, or length restrictions), Fluent API offers a more concise and readable approach than multiple data annotations. It also allows for configurations not supported by data annotations, such as configuring computed columns.
- **Separation of Concerns:** Fluent API allows a clear separation of concerns by keeping domain models (entity classes) free of data annotations. This is useful if we want to keep our model classes clean and ensure that all database-related configurations are centralized in the `OnModelCreating` method of the `DbContext`.

So, Fluent API configurations provide flexibility, control, and the ability to configure more complex EF Core configurations that are either unsupported or difficult to implement with data annotations.

In the next article, I will discuss [Relationships in Entity Framework Core using Fluent API](#) with Examples. In this article, I explain how to Configure Fluent API in Entity Framework Core with Examples. I hope you enjoyed this Fluent API configuration in the EF Core article.